Repair ledger — Evidence Reasoning Demo

Issues found and fixed during the build, in Project Witness's "make claims, attack
claims, repair claims, narrow claims" spirit. Newest first.

R1 — FIRST BUILD (v0.1.0): Evidence Reasoning Demo

First demo build — Move 14 of Project Witness, and the first component that drives
the whole local chain end to end. It takes one local evidence file, runs frozen
Aletheia (bundle + verify), frozen Bridge (convert + verify), and frozen ODIN
(reason + verify), preserves each component's output exactly inside the demo folder,
and writes a deterministic demo report, step record, and checksums file that can be
verified later. It is not connected to Companion or Coordinator.

Scope held deliberately narrow: one evidence file; one bundle; one bridge conversion; one ODIN reasoning run; one demo report. No multiple files, directories, web, sensors, OCR, LLM, natural language, truth claims, action gating, signing, or anchoring. The demo proves the chain executes and verifies — it never proves the evidence contents are true.

Design decisions worth recording:

- **Component outputs preserved, never rewritten.** The frozen components write directly into the demo's `aletheia_bundle/`, `bridge_output/`, and `odin_output/` subfolders; the demo reads them (manifest, bridge report, reasoning report) but never edits them, and references them by hash in `demo_checksums.txt`.
- **Ordered short-circuit with a precise demo_status.** The six steps run in a fixed order; the first failure raises a typed `StepError` carrying the exact `demo_status` (e.g. `FAILED_BRIDGE_CONVERT`) and stops the chain, so later steps never run. The step record marks the failing step `FAILED` and the rest `SKIPPED_AFTER_FAILURE`. Tested for every step boundary.

- **Truth-claim guard at verify.** `verify` confirms `generated_claims` contains only `evidence.present` / `evidence.integrity_verified` and refuses any truth-like predicate (token scan), and additionally scans the bridge report's surfaced predicates — so a hand-injected `contents.true` is caught (`DISALLOWED_PREDICATE` / `TRUTH_PREDICATE`) even after the checksums are resynced.
- **Verify recomputes cross-references rather than trusting the report.** It checks `case_id` across the demo report, demo steps, bridge report, and ODIN report; `evidence_id` across the Aletheia manifest and bridge report; that `odin_reasoning_status == CONSISTENT` (in both the demo report and the ODIN report); and `demo_status == DEMO_COMPLETED` — each after a canonical + hash check, so resyncing the checksums after a lie does not help.
- **Frozen boundary is injectable.** The component runner calls the real frozen CLIs via subprocess at runtime, but the unit suite injects an in-process fake runner that writes realistic component outputs, so

the freeze gate spawns no subprocesses and does not require the real frozen folders. The subprocess path is exercised only by the documented manual integration smoke test.

- **Reused the stack's honest patterns:** deterministic canonical JSON; atomic writes (`os.replace`); refuse a non-empty output directory; refuse missing / directory / symlink evidence sources (size left to frozen Aletheia); `require()` not bare assert; strict checksums; unknown-top-level-file rejection; in-process selfcheck that never launches the unit suite.

Real end-to-end integration confirmed. Against frozen Aletheia v0.1, Bridge v0.1, and ODIN v0.1: a real evidence file produced `DEMO_COMPLETED` with `aletheia_status = VERIFIED_PRESENT`, `bridge_status = CONVERTED`, `odin_reasoning_status = CONSISTENT`, exactly `evidence.present` + `evidence.integrity_verified`, all three component outputs preserved in their subfolders, and `demo verify` passed.

Release gate (Coordinator v0.1.8 / Companion v0.1 / Aletheia v0.1 / ODIN v0.1 / Bridge v0.1 lesson applied directly): no subprocess

orchestrator in the freeze
gate. The authoritative gate is the explicit command
list in RELEASE_COMMANDS.md
plus the static `verify_release_manifest.py`
(launches no subprocesses; additionally
checks the README says the chain executes and
verifies, disclaims proving truth, and
states it is not connected to Companion or
Coordinator yet).

Verified: all release-gate commands pass individually
from a clean extraction
(see TEST_RESULTS.md); selfcheck 8/8 in-process
gates; verify_release_manifest.py
passes (9/9, no subprocesses); every step-failure
boundary short-circuits with the
correct demo_status; tamper to any demo file, the
checksums, the file/subfolder set,
a referenced component file, the cross-referenced
IDs/statuses, or an injected truth
predicate is detected with the correct reason.